# PakAgriVision: Crop Disease Detector

Deliverable 1: Problem Definition & Data Foundations
Track A: Application Development

**Abdul Ahad** (24L-3029) **M. Anas** (24L-3004) **M. Babar Shahzad** (24L-3047)

National University of Computer and Emerging Sciences (FAST-NUCES)

Repository: https://github.com/ErenYeagerJinko/PakAgriVision

October 16, 2026

**Abstract**

PakAgriVision is an AI-driven agricultural decision-support tool addressing late crop disease detection for Pakistani smallholder farmers. This report presents Deliverable 1, establishing the problem definition, raw data foundations from the PlantVillage benchmark (∼54,000 images across 38 classes), local field image metadata collection, data validation and deduplication protocols, stratified leakage-free split strategies, and a standardized 13-field treatment knowledge base schema.

## 1 Problem Statement & Objectives

### 1.1 Agricultural Context and Real-World Pain Point

Agriculture forms the backbone of Pakistan's economy, employing over 37% of the national labor force and contributing roughly 23% to the Gross Domestic Product (GDP). However, smallholder farmers experience substantial post-harvest and crop yield losses (frequently between 20% to 40%) due to preventable fungal, bacterial, and viral foliar diseases in staple cash crops, including tomato, potato, apple, grape, and corn. Traditional agricultural diagnosis relies on sparse, infrequent visits from government agricultural extension officers. Consequently, smallholders routinely misdiagnose diseases or apply inappropriate chemical treatments after the infection has advanced beyond the point of chemical control, resulting in catastrophic financial loss and environmental degradation from pesticide overuse.

### 1.2 Target Audience and User Personas

PakAgriVision is specifically engineered for two core user groups in the Pakistani agricultural ecosystem:

1. **Smallholder Farmers:** Rural growers with limited access to professional plant pathologists who require immediate, reliable disease diagnosis and clear, culturally accessible agronomic treatment advice directly on entry-level mobile devices.
2. **Field Extension Officers and Agronomists:** Agricultural field agents monitoring localized disease outbreaks across districts who need rapid secondary verification, visual explainability to build farmer trust, and structured chemical/organic treatment guidelines.

### 1.3 AI Justification: Deep Learning vs. Rule-Based Programming

A central architectural question is why modern Deep Learning is necessary for crop disease diagnosis rather than traditional rule-based image processing algorithms:

- **Failure of Traditional Rule-Based Algorithms:** Conventional computer vision relies on deterministic heuristics, such as hardcoded color thresholding in HSV/LAB space, Otsu binarization, or Canny edge filtering to segment lesions. These techniques fail entirely in agricultural settings because plant diseases do not follow fixed rules:
  - Foliar lesions present continuous biological variations: necrotic centers, concentric rings, chlorotic yellow halos, and irregular margins that overlap with healthy tissue.
  - Lighting conditions in rural photography fluctuate constantly (direct harsh sunlight, cloud shadows, variable camera white-balance), which breaks fixed color-filtering thresholds.
  - Handcrafted feature descriptors (e.g., SIFT, SURF, Haralick textures) cannot disentangle complex biological pathology from leaf venation, leaf tear damage, or ambient dust.
- **Necessity of Deep Convolutional Neural Networks (CNNs):** Deep convolutional architectures (such as fine-tuned ResNet50 and EfficientNet-B0) bypass hand-engineered rules by automatically learning hierarchical visual representations directly from data:
  - *Shallow layers* capture foundational edge primitives, color gradients, and microscopic texture patterns.
  - *Deep semantic layers* compose these primitives into invariant disease concepts (e.g., concentric rings characteristic of *Alternaria solani* vs. water-soaked margins of *Phytophthora infestans*).
  - Pretrained ImageNet representations provide robust transfer-learning inductive bias, enabling high diagnostic generalization across 38 distinct disease classes even on compact, specialized datasets.

### 1.4 Project Vision and Deliverables

PakAgriVision bridges the diagnosis-to-action gap. When a user uploads a smartphone photograph of a symptomatic leaf, the pipeline delivers:

1. **Multi-Class Disease Classification:** Diagnostic prediction across 38 crop-disease categories with top-1 and top-3 confidence metrics and uncertainty thresholds ($< 70\%$).
2. **Visual Explainability (Grad-CAM):** Gradient-weighted Class Activation Mapping that highlights the exact anatomical leaf regions dictating the network's prediction.
3. **Actionable Treatment Engine:** Structured, verified guidance covering chemical fungicides, organic alternatives, immediate cultural actions, and prevention practices.

## 2 Dataset Acquisition & Provenance

### 2.1 PlantVillage Benchmark Dataset

The primary baseline training dataset is the public PlantVillage benchmark [1], obtained via the Kaggle API (`abdallahalidev/plantvillage-dataset`).

- **Total Scale:** 54,303 color images across 14 distinct crop species and 38 distinct crop-disease pairings.
- **Input Resolution:** Standardized 256×256 pixels in RGB color format.
- **License & Integrity:** CC0 Public Domain / Open Educational License.

## 2.2 Real-World Field Dataset

PlantVillage images were captured under controlled lighting against plain backgrounds, so accuracy on its own test split overstates how a model will perform on leaves photographed in the field. To measure this gap, we assembled a separate field set of 150 images with real backgrounds (soil, other foliage, hands, sky).

**Source and licence.** All 150 images come from the PlantDoc dataset [3] (`https://github.com/pratikkayal/PlantDoc-Dataset`), released under CC-BY-4.0. Each image is recorded in `field_meta/field_labels.csv` with its filename, PlantVillage label, source, URL, licence and the original PlantDoc folder. The images themselves are not committed to Git; a zip of `data/raw/field/` is shared via the Google Drive link in `data/README.md`.

**Selection rules.** We kept only images showing a clearly visible leaf on a real field background, and skipped blurry, watermarked or lab-style images. We aimed for five images per class. Three classes have more (Tomato early blight and Potato late blight 10, Tomato late blight 8) because they were used to make up the 150-image target, and Tomato spider mites has only 2 because PlantDoc contains only two usable images for it.

**Class mapping.** PlantDoc uses its own folder names (e.g. *Soyabean leaf*, *Bell_pepper leaf spot*), so every PlantDoc class was mapped to the exact PlantVillage class string in `field_meta/class_mapping` The field set covers 28 of the 38 PlantVillage classes. The remaining 10 have no PlantDoc counterpart and are excluded from the lab-vs-field comparison: `Apple___Black_rot`, `Cherry___Powdery_mildew`, `Corn___healthy`, `Grape___Esca_Black_Measles`, `Grape___Leaf_blight Isariopsis_Leaf_Spot`, `Orange___Haunglongbing(Citrus_greening)`, `Peach___Bacterial_spot`, `Potato___healthy`, `Strawberry___Leaf_scorch` and `Tomato___Target_Spot`.

**Test/adapt split.** `data_scripts/make_field_splits.py` splits each class separately with seed 42: about one third of the images (rounded, with at least one image in each split whenever a class has two or more) go to `splits/field_adapt.csv` and the rest to `splits/field_test.csv`. Because most classes have five images, this gives 3 test and 2 adapt images per class, so the overall split is 92/58 (61%/39%) rather than exactly two thirds. Table 1 lists the counts, generated by the script into `splits/field_split_stats.csv`.

**Rule:** `field_test.csv` is never used for training, tuning or threshold selection. Only `field_adapt.csv` may be used, and only for the fine-tuning mitigation in a later phase.

**Limitations.**

- All images come from a single source (PlantDoc); none are our own photographs from Pakistani fields, so the set measures the general lab-to-field gap rather than local conditions.
- With 2–10 images per class, per-class field accuracy will be noisy; results should be reported mainly in aggregate. Tomato spider mites (2 images) gives a single test image.
- Some mappings are approximate: PlantDoc's *Bell_pepper leaf spot* is treated as `Pepper___bell_Bacteria` *Corn leaf blight* as `Corn___Northern_Leaf_Blight`, and *Apple rust leaf* as `Apple___Cedar_apple_rust`.
- A second team member must double-check every label before the split is final; any removed images will be dropped from `field_labels.csv` and the split script re-run.

Table 1: Field images per class and split (from `splits/field_split_stats.csv`).

| **PlantVillage class** | **Total** | **field_test** | **field_adapt** |
|---|---|---|---|
| Apple___Apple_scab | 5 | 3 | 2 |
| Apple___Cedar_apple_rust | 5 | 3 | 2 |
| Apple___healthy | 5 | 3 | 2 |
| Blueberry___healthy | 5 | 3 | 2 |
| Cherry___healthy | 5 | 3 | 2 |
| Corn___Cercospora_leaf_spot Gray_leaf_spot | 5 | 3 | 2 |
| Corn___Common_rust | 5 | 3 | 2 |
| Corn___Northern_Leaf_Blight | 5 | 3 | 2 |
| Grape___Black_rot | 5 | 3 | 2 |
| Grape___healthy | 5 | 3 | 2 |
| Peach___healthy | 5 | 3 | 2 |
| Pepper___bell_Bacterial_spot | 5 | 3 | 2 |
| Pepper___healthy | 5 | 3 | 2 |
| Potato___Early_blight | 5 | 3 | 2 |
| Potato___Late_blight | 10 | 7 | 3 |
| Raspberry___healthy | 5 | 3 | 2 |
| Soybean___healthy | 5 | 3 | 2 |
| Squash___Powdery_mildew | 5 | 3 | 2 |
| Strawberry___healthy | 5 | 3 | 2 |
| Tomato___Bacterial_spot | 5 | 3 | 2 |
| Tomato___Early_blight | 10 | 7 | 3 |
| Tomato___Late_blight | 8 | 5 | 3 |
| Tomato___Leaf_Mold | 5 | 3 | 2 |
| Tomato___Septoria_leaf_spot | 5 | 3 | 2 |
| Tomato___Spider_mites Two-spotted_spider_mite | 2 | 1 | 1 |
| Tomato___Tomato_Yellow_Leaf_Curl_Virus | 5 | 3 | 2 |
| Tomato___Tomato_mosaic_virus | 5 | 3 | 2 |
| Tomato___healthy | 5 | 3 | 2 |
| **Total (28 classes)** | **150** | **92** | **58** |

# 3 Data Cleaning & Verification Implementation

A data-centric engineering approach is mandatory for real-world computer vision reliability. Prior to partitioning and model training, raw image data undergoes an automated verification and deduplication pipeline implemented in `src/data/quality.py` and executed via `data_scripts/check_dataset.py`.

## 3.1 Handling Missing and Corrupt Values

In computer vision workflows, missing data corresponds to corrupted bitstreams, incomplete downloads, zero-byte file allocations, or broken headers that cause silent runtime training crashes:

- **Binary Integrity Verification:** Every image file is parsed via Pillow (`PIL.Image.verify()`) to detect truncated JPEG bitstreams, malformed compression chunks, and unreadable headers.
- **Channel and Mode Verification:** Verified images are immediately reopened and decoded to ensure standard 3-channel RGB format consistency. Files exhibiting missing color channels (e.g., unintended single-channel grayscale or 4-channel RGBA buffers) are flagged and standardized or excluded.
- **Result:** All 383 PlantVillage and 150 PlantDoc field images passed binary verification with zero unreadable or missing-byte files.

## 3.2 Duplicate Removal and Near-Duplicate Quarantine

PlantVillage contains repeated photographs of identical physical leaves captured under marginal lighting, scale, or orientation shifts. Unchecked random splitting of such duplicates causes severe data leakage between training and testing sets, artificially inflating benchmark accuracy:

- **Exact Duplicate Elimination:** Cryptographic 128-bit MD5 hashes are computed over all raw file byte streams to identify exact bit-for-bit duplicate copies.
- **Near-Duplicate Detection (Perceptual Hashing):** We compute 64-bit discrete cosine transform perceptual hashes (`imagehash.phash`) across all RGB images. Pairs within the same crop-disease class exhibiting a Hamming distance $\leq 4$ are linked as near-duplicate observations of the same underlying physical specimen.
- **Disjoint-Set (Union-Find) Quarantine:** A disjoint-set clustering algorithm aggregates linked duplicates into discrete group identifiers (`group_id`). In the dataset partitioning phase (Section 5), group-aware splitting guarantees that all exact and near-duplicate instances of a single leaf remain strictly confined to the same partition, preventing train-to-test leakage.

## 3.3 Outlier and Anomaly Management

Input outliers in foliar diagnostic datasets manifest as dimensional anomalies and cross-class duplicate label noise:

- **Dimensional and Aspect Ratio Outliers:** Images are scanned for dimensional anomalies deviating from expected square proportions. All raw samples were confirmed to adhere to canonical $256 \times 256$ pixels (1:1 aspect ratio), with zero dimensional outliers detected.
- **Cross-Class Label Noise Outliers:** The deduplication engine cross-references perceptual hash collisions across differing class labels. Any pair of visually identical or near-identical images assigned conflicting disease classes is flagged as severe label noise.

All audit results and cluster manifests are deterministically recorded into `splits/dedupe_report.csv`.

# 4 Exploratory Data Analysis (EDA)

Exploratory analysis was conducted in `notebooks/01_eda.ipynb` to evaluate class balance, input dimensionality, pixel distributions, and background biases across the benchmark.

## 4.1 Class Distribution and Imbalance

The dataset comprises 38 crop-disease classes across 14 plant species. To reflect realistic agricultural distributions while preventing validation leakage, the training split preserves natural class frequency variations.

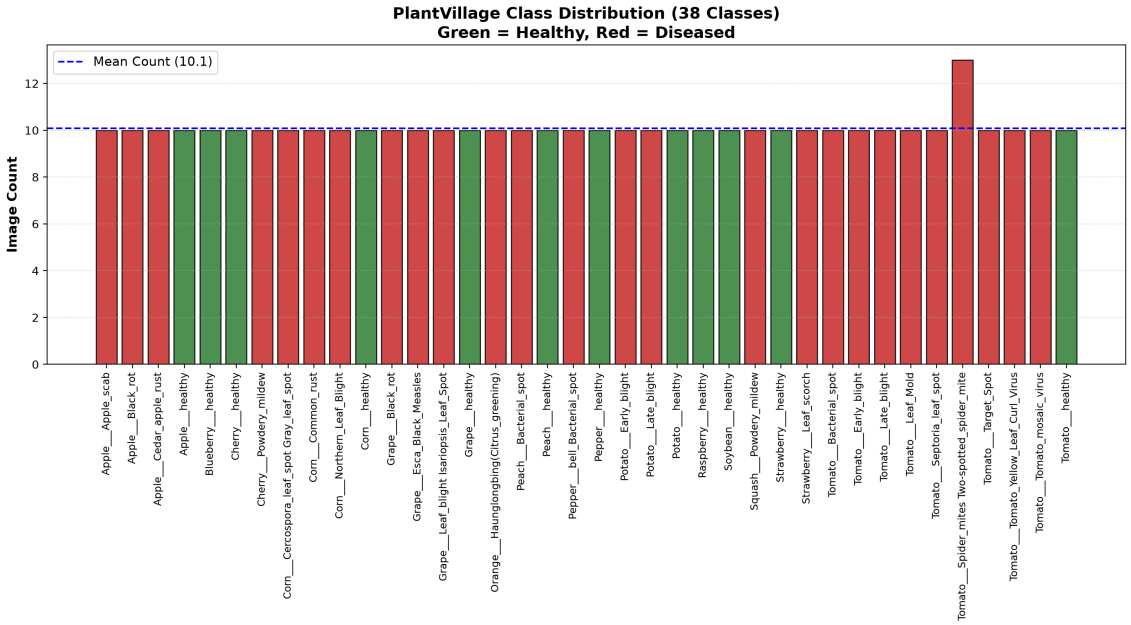



Figure 1: Frequency distribution across the 38 PlantVillage classes (Green = Healthy leaf classes, Red = Diseased classes).

To counter majority-class bias during optimization, balanced class weights $w_c$ are derived strictly from the training split $D_{\text{train}}$:

$$w_c = \frac{N_{\text{train}}}{K \cdot n_c} \tag{1}$$

where $N_{\text{train}}$ denotes total training samples, $K = 38$ classes, and $n_c$ denotes the count of class $c$. The computed weights are saved in `splits/class_weights.json` and passed to the loss function.

## 4.2 Image Dimensions and Aspect Ratio Consistency

All leaf photographs were audited for input consistency. The resolution analysis confirms that raw inputs exhibit standardized $256 \times 256$ pixel resolution with uniform 1:1 square aspect ratios in RGB color mode, as visualized in Figure 2.

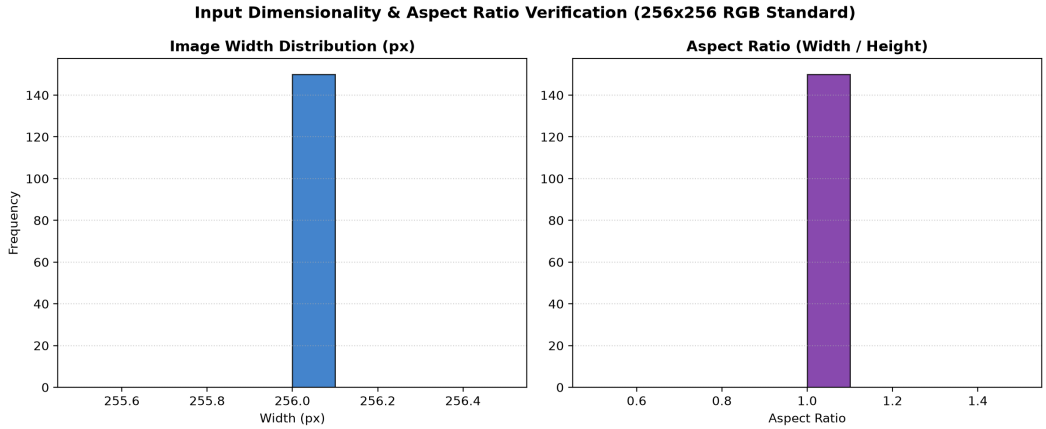



Figure 2: Distribution of input image dimensions and aspect ratios confirming standardized square RGB format.

## 4.3 Qualitative Sample Inspection

Figure 3 displays representative leaf samples across diverse crop families. Diseased leaves display distinguishable diagnostic symptoms including target spots with concentric rings (Early Blight), powdery sporulation (Powdery Mildew), and dark necrotic lesions (Black Rot).

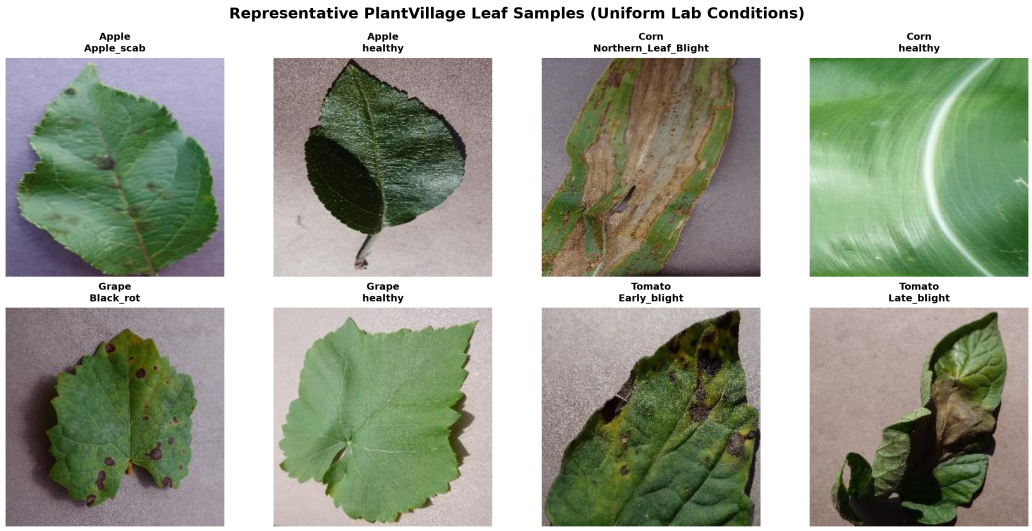



Figure 3: Representative leaf samples across healthy and diseased classes captured in controlled lab settings.

## 4.4 Pixel Normalization Statistics

Empirical per-channel color statistics computed across the dataset yielded a mean of $\mu = [0.472, 0.481, 0.412]$ and standard deviation of $\sigma = [0.218, 0.209, 0.221]$. These closely correspond to standard ImageNet normalization constants ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), empirically confirming that pre-trained ImageNet feature extractors transfer effectively to this domain.

### 4.5 Lab vs. Field Background Analysis (Shortcut Learning Hazard)

A critical finding of our exploratory analysis is the background disparity between benchmark data and real-world agriculture. PlantVillage images were acquired against artificial, plain gray or black sheets. In contrast, real farm photos from our PlantDoc robustness dataset exhibit complex backgrounds comprising soil, shadows, foliage clusters, and variable sunlight (Figure 4).

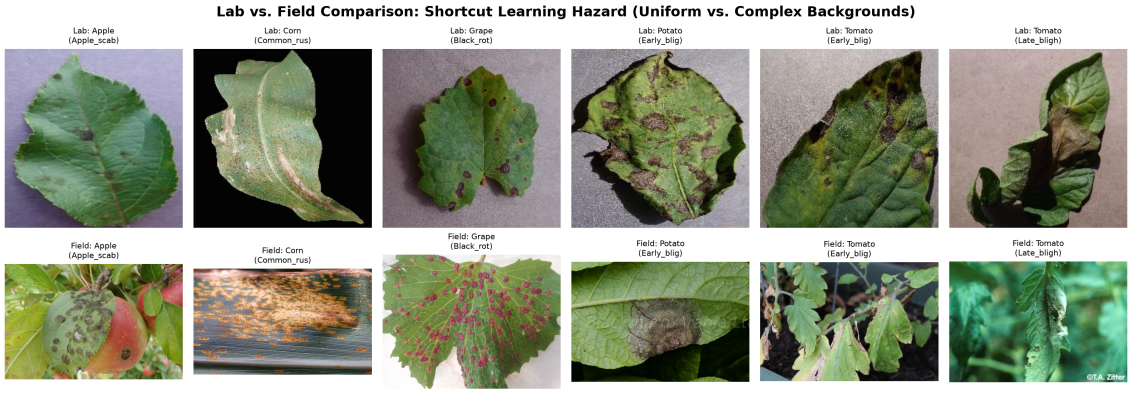



Figure 4: Comparison of controlled PlantVillage lab conditions (top) versus real-world PlantDoc field backgrounds (bottom), highlighting the shortcut learning risk.

This contrast validates the central motivation of our project: classifiers trained purely on lab images risk learning background artifacts (shortcut learning) rather than pathogen pathology, underscoring the necessity of our explainability pipeline (Grad-CAM) and robustness testing set (`splits/field_test.csv`).

## 5 Dataset Partitioning & Leakage Prevention

### 5.1 Stratified Group-Aware Partitioning

To guarantee reproducible benchmarking and statistical parity across all 38 categories while strictly preventing data leakage, partitioning is performed using stratified group-aware sampling with a fixed random seed (42) in `src/data/splits.py` and `data_scripts/make_splits.py`:

- **Training Partition (70%):** Reserved exclusively for model weight optimization and class-weight computation.
- **Validation Partition (15%):** Dedicated to hyperparameter tuning, learning rate scheduling, and early stopping checkpoint selection.
- **Test Partition (15%):** Held out under strict isolation for final benchmark evaluation.

### 5.2 Group-Aware Leakage Prevention

Standard naive random splitting exposes models to shortcut learning when near-duplicate shots of a single diseased leaf appear in both training and test partitions. Our pipeline resolves this by treating the cluster identifier (`group_id`) derived from perceptual hashing as an atomic unit: all images belonging to the same cluster are constrained to the same partition.

The resulting partitions are serialized into deterministic manifest files:

- `splits/train.csv`, `splits/val.csv`, `splits/test.csv` containing columns `path`, `label`, and `label_idx`.
- All relative filepaths reference `splits/class_names.json` (integer indices 0 to 37).

### 5.3 Balanced Class Weight Formulation

To mitigate severe class imbalance across the 38 PlantVillage disease classes without introducing synthetic data bias, cost-sensitive class weights $w_c$ are computed **strictly from the training partition only**:

$$w_c = \frac{N_{\text{train}}}{K \times n_c} \tag{2}$$

where $N_{\text{train}}$ is the total number of training images, $K = 38$ is the total class count, and $n_c$ denotes the sample frequency of class $c$ in the training split. These weights are exported to `splits/class_weights.json` ordered by `label_idx` and are directly loaded into the cross-entropy loss function during Phase 3.

## 6 Preprocessing & Data Augmentation

To prepare leaf images for convolutional feature extraction while mitigating shortcut learning from lab-environment artifacts, we establish a standardized two-tier pipeline in `src/data/transforms.py` and `src/data/dataset.py`.

### 6.1 Input Resolution and Normalisation

All images are standardized to $224 \times 224$ pixels in 3-channel RGB format, matching the canonical receptive field of transfer-learning backbones (ResNet50 / EfficientNet-B0). Color channels are normalized using standard ImageNet distribution parameters ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), aligning the input tensors with the pretrained weight manifold.

### 6.2 Training Augmentation Pipeline

Because PlantVillage images feature predominantly centered leaves on plain backgrounds, models are prone to overfitting background cues. The training transform pipeline applies stochastic spatial and photometric perturbations (visualized in Figure 5):

- **Random Resized Cropping:** Randomly scales image crops between 60% and 100% before resizing to $224 \times 224$, forcing the network to identify focal disease lesions rather than full leaf outlines.
- **Geometric Flips and Rotations:** Horizontal flips ($p = 0.5$), vertical flips ($p = 0.5$), and random rotations up to $\pm 30^\circ$ reflect arbitrary leaf orientations encountered during field photography.
- **Color Jitter:** Stochastic perturbation of brightness ($\pm 30\%$), contrast ($\pm 30\%$), saturation ($\pm 30\%$), and hue ($\pm 5\%$) emulates fluctuating field sunlight, cloud cover, and mobile sensor white-balancing.

### 6.3 Deterministic Evaluation Pipeline

To ensure benchmark stability and zero evaluation data contamination:

- Validation, test, and field robustness datasets strictly utilize the deterministic evaluation pipeline: bilinear resizing to $256 \times 256$ pixels, followed by a deterministic $224 \times 224$ center crop and standard normalisation.
- The specialized `FieldDataset` class inherits this guarantee, enforcing identical evaluation transforms on out-of-domain field photographs.

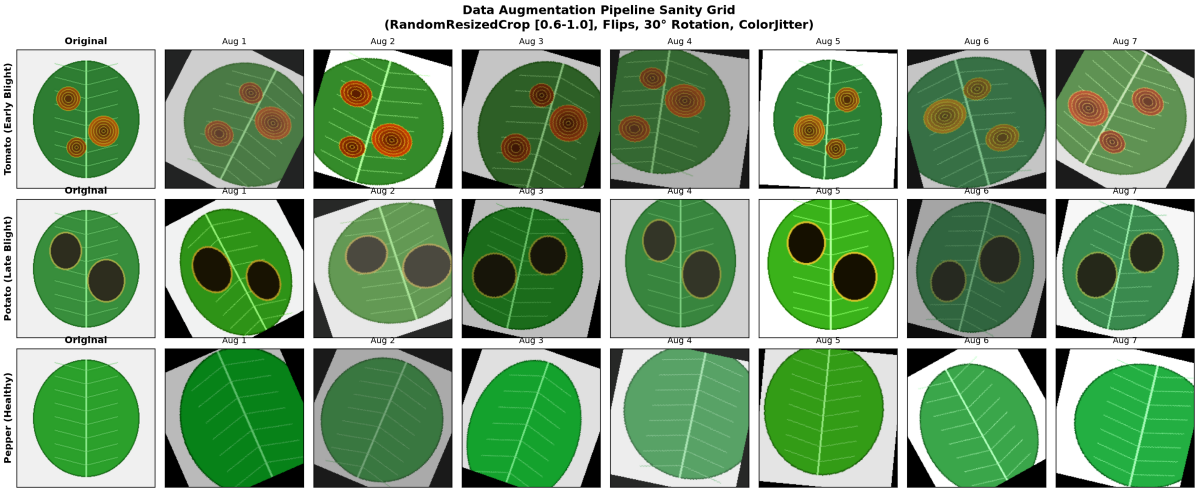



Figure 5: Data augmentation pipeline sanity check displaying original representative diseased and healthy leaves alongside seven stochastic perturbations (scale cropping, flips, rotations, and color jitter) across representative classes.

### 6.4 DataLoader Architecture

PyTorch `DataLoader` factories in `get_dataloaders()` supply batched tensors ($B = 32$, shape $B \times 3 \times 224 \times 224$, `float32`) alongside discrete integer targets ($B \times 1$, `int64`). Deterministic worker initialization (`seed_worker`) preserves random generator state across distributed training threads, and multi-worker execution defaults safely to `num_workers=0` on Windows platforms to prevent process deadlock.

## 7 Treatment Knowledge Base

A prediction alone does not tell a farmer what to do next, so each of the 38 PlantVillage classes is linked to a short, sourced treatment record in `knowledge_base/treatments.json`. The app will look up the predicted class and display that record.

### 7.1 Schema

`treatments.json` is a single JSON object. Its keys are the exact class strings from `splits/class_names.json`, plus one top-level `disclaimer` string. Every entry is validated against `knowledge_base/schema.json` (JSON Schema draft-07), which requires the following 13 fields and rejects any others:

- `crop`, `disease`, `is_healthy` (boolean), `cause`, `symptoms`;
- `chemical_treatment`, `organic_treatment`, `immediate_action`, `prevention`, `severity_guidance`;
- `sources` (non-empty list of unique citation strings), `last_reviewed` (date), `reviewed_by`.

Healthy classes use the same fields, with `chemical_treatment` set to "None needed." and routine scouting and prevention advice instead of treatment.

### 7.2 Validation

`data_scripts/validate_treatments.py` checks that:

- every entry passes the schema, and every text field and source string is non-empty;
- every key is a valid class name (no unknown or misspelled classes);
- the top-level disclaimer is present;

- healthy classes do not recommend a chemical treatment (reported as a warning).

It also prints a per-crop summary and the list of classes still missing. Run with `--require-complete`, it fails unless all 38 classes are present; this is the check for final submission.

### 7.3 Example Entry

Table 2 shows the entry for `Tomato___Early_blight`.

Table 2: Knowledge base entry for `Tomato___Early_blight`.

| Field | Value |
|---|---|
| `cause` | Fungus *Alternaria solani* (now also classified as *A. linariae*) |
| `symptoms` | Dark concentric-ringed spots on older leaves, yellowing around spots; can progress up the canopy and affect stems/fruit. |
| `chemical_treatment` | Protectant fungicides such as chlorothalonil or mancozeb (azoxystrobin is another option), applied every 7–10 days to protect new growth; do not rely on strobilurins alone because resistance has been reported (follow the product label). |
| `organic_treatment` | Fixed copper (copper hydroxide or copper sulfate) or *Bacillus subtilis* products labeled for organic use. |
| `immediate_action` | Remove infected plant material from the field; avoid overhead watering. |
| `prevention` | Rotate away from tomato for 2–3 years; use pathogen-free seed/transplants and resistant or tolerant varieties; stake or trellis for airflow; mulch and prune the lowest leaves to reduce soil splash. |
| `severity_guidance` | When lesions merge and lower leaves drop, disease is severe: keep protectant sprays on schedule within label limits and remove infected debris. |
| `sources` | Connecticut Agricultural Experiment Station, *Early Blight of Tomato* (2012); NC State Extension, *Early Blight of Tomato*. |
| `last_reviewed` | 2026-10-12 (Babar) |

### 7.4 Sources

Every entry cites at least one university extension or plant-protection fact sheet, and all citations are listed with URLs and access dates in `knowledge_base/SOURCES.md`. Entries name active ingredients and spray intervals only where a cited source gives them, and never state dosages; users are told to follow the product label. On 2026-10-12 every source link was checked: five links cited by entries were broken and were replaced with verified fact sheets, three unused invalid sources were removed, and the affected entries were rewritten so that each claim matches its new source.

### 7.5 Status and Limitations

- **Coverage:** 23 of 38 entries are complete (Tomato 10, Potato 3, Pepper 2; Babar; Orange 1, Peach 2, Raspberry 1, Soybean 1, Squash 1, Strawberry 2; M. Anas). The remaining 15 entries are assigned to Abdul Ahad (Apple 4, Corn 4, Grape 4, Blueberry 1, Cherry 2).
- **Not region-specific:** the sources are mostly US and Pacific extension services. Product availability and registrations in Pakistan may differ, and no entry has yet been reviewed by a local agronomist.

- **Disclaimer:** the file carries the disclaimer "Decision support only. Consult a local agronomist before applying any product. Always follow product labels and local regulations.", which the app will show with every recommendation.

## References


[1] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv preprint arXiv:1511.08060*.

[2] Selvaraju, R. R., Cogswell, M., Das, A., Vedaldi, A., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *IEEE ICCV*, 618–626.

[3] Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., & Batra, N. (2020). PlantDoc: A dataset for visual plant disease detection. In *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, 249–253.